

A wizard, ray-marched

Rendering with distance functions, in Futhark, on WebGPU

Techniques for programmers who haven’t done graphics

What we built

- A wizard on a floating rock, flying over endless terrain with rivers, a sea, cottages, towers and pyramids
- Every pixel is computed by a **pure function**: no meshes, no triangles, no game engine
- Written in **Futhark**, a functional array language, compiled to **WebGPU** compute shaders
- Live in the browser: hot reload, a parameter drawer, free flight, spells, a game controller
- 1080p at 60 fps on a laptop GPU



Everything here, from the beard to the clouds, is maths evaluated per pixel.

The usual way, and ours

Rasterisation (games, most 3D)

- Shapes are lists of triangles
- The GPU projects each triangle onto the screen and fills its pixels
- Fast, but every shape must first be modelled as a mesh

Ray marching (this project)

- Shapes are **functions** $d(p)$: “how far is point p from the surface?”
- For each pixel, shoot a ray from the camera and walk along it until it hits something
- Shapes are a few lines of maths; blending, bending and repeating them is trivial

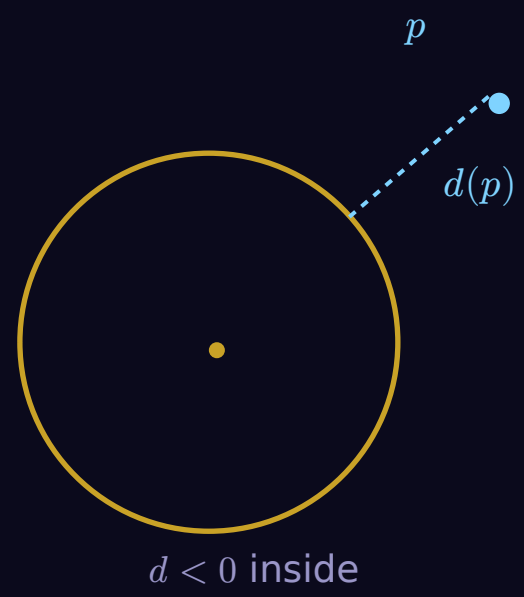
The idea is old (Hart, 1996, “Sphere tracing”); it became popular through the demoscene and Shadertoy.

Signed distance functions

A **signed distance function** (SDF) returns the distance from p to the nearest surface: positive outside, negative inside, zero on it.

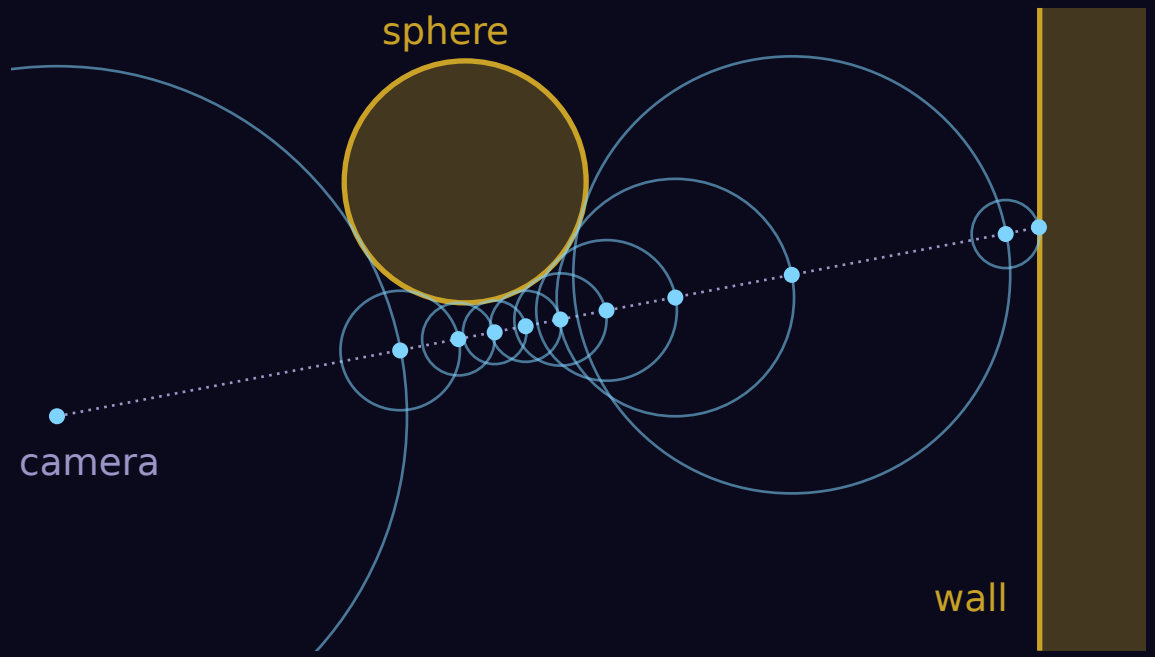
```
def sd_sphere p r = length p - r

-- a segment a-b, thickened by r
def sd_capsule p a b r =
  let pa = p - a
  let ba = b - a
  let h = clamp 0 1 (dot pa ba / dot ba ba)
  in length (pa - scale h ba) - r
```



A scene is the **minimum** of its parts' distances: the nearest surface wins.

Sphere tracing



The distance $d(p)$ is a **safe step**: nothing is closer than that, so we can jump that far along the ray without passing through anything.

```
loop t = 0 while t < tmax do
  let d = scene (ro + t * rd)
  in if d < eps * t then hit t
     else t + d
```

Big steps in empty space, tiny ones near surfaces. A few dozen steps per pixel usually suffice.

Combining shapes

- **Union**: `min a b` (the nearer surface)
- **Subtraction**: `max a (-b)` (carve b out of a)
- **Smooth union**: blend two shapes with a fillet of radius k. This is how the robe flows into the shoulders and the arms into the robe:

```
def smin k a b =
  let h = clamp 0 1 (0.5 + 0.5 * (b - a) / k)
  in mix b a h - k * h * (1 - h)
```

- Each part also carries a **material id**, so the nearest one decides the colour

- **Transform space, not shapes**: to move, rotate or bend a shape, transform the point before asking its SDF
  - the hat droops: `sd_cone (bend droop p)`
  - the head turns: `sd_head (rotate (-look) p)`
- **Repetition**: fold space into a cell so one shape becomes many (`fract`, angular sectors). The orb's cradle prongs and the rune circle are folded this way
- Distortions break the "safe step" promise, so we scale the distance down by the worst stretch

Normals, shadows and ambient occlusion: all from d

Normal = gradient of d, by finite differences. A tetrahedron of 4 samples instead of 6:

```
n = normalise (Σ k · d(p + e·k))
    for k in 4 tetrahedron corners
```

Soft shadows: march towards the light; how close the ray grazes other surfaces ($\frac{d}{t}$, smallest along the way) gives a penumbra for free.

Ambient occlusion: sample d a few times along the normal. If it is smaller than the distance travelled, something is nearby blocking the sky: darken.

All three only need the same function d. No shadow maps, no light probes.

Lighting then is classic: diffuse (Lambert) + specular highlight (Blinn) + rim light + fill, per light: the moon, the orb, three potions.

Textures without images

- Every pattern is procedural: **hashes** give repeatable randomness, **value noise** interpolates it, **fbm** sums octaves of it
- Bark, cloth weave, leather stitching, beard locks, hat stars, bricks
- **Bump mapping**: a height pattern tilts the normal (its gradient) without changing the geometry
- **Footprint filtering**: a pixel at distance t is $\approx t \cdot$ (angle per pixel) wide; patterns finer than that fade out instead of flickering



Bricks: cells with every other row offset, a hash per brick for its shade, mortar as a bump.

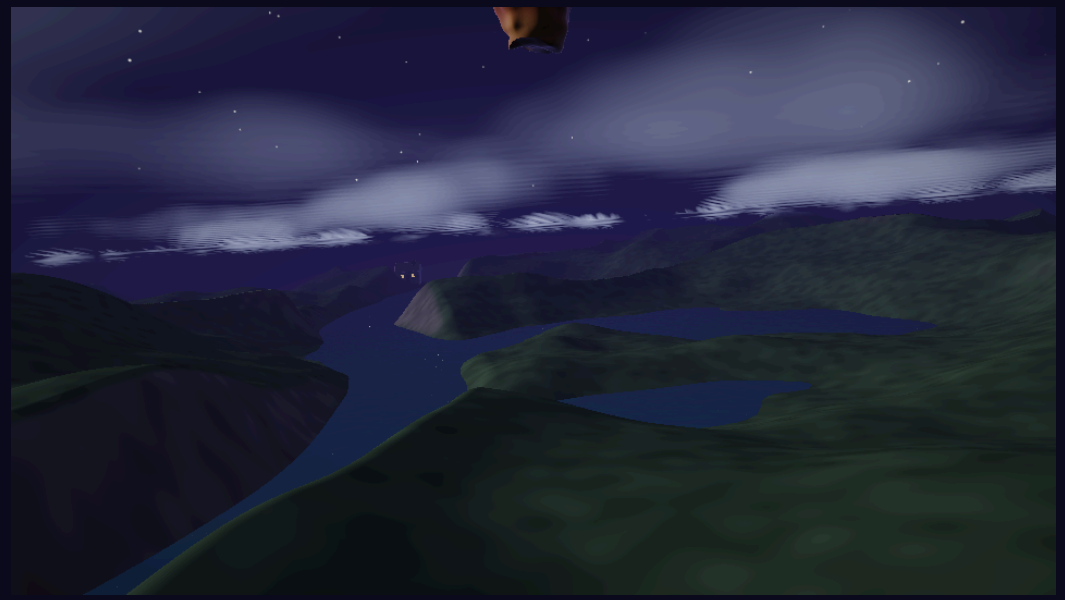
Glass, glow and clouds

- **The orb** is solved analytically (ray–sphere), and the ray **refracted** in and out (Snell's law), with a **Fresnel** reflection
- **Glows** are **media**: each march step adds light by a density, e.g. $e^{-d/\,\mathrm{extent}}$ near the orb; steps are capped so thin glows aren't skipped
- **Lightning**: jagged polylines; distance to them gives a hot core and a halo
- **Clouds**: a slab between two heights, integrated with a fixed sample budget apart from the march



Terrain: a heightfield, carefully

- Height $h(x, z)$ = sum of noise octaves (fbm), plus ridges $1 - |2f - 1|$
- A heightfield is not a distance, but its slope is bounded, so $0.6(y - h(x, z))$ is a safe underestimate
- **Analytic normals**: the noise is differentiated alongside its value (chain rule), so one evaluation gives the slope instead of four
- **Rivers** carve along a contour of another noise, with the gradient carried through the carving
- **A cheap upper bound** (first octaves only) stands in for the full height while far above the ground



The wizard stays at the origin; the world moves past him through a “flight frame” (position, heading, bank).

Infinite buildings: hashing a grid

- The world is divided into 30-unit cells; a hash of the cell's coordinates decides whether it has a building, which kind, where and how rotated
- The building's distance is exact nearby, but marching every cell would be ruinous. Instead, **conservative bounds**:
  - above every roof: height above the tallest one
  - other cells' buildings sit inside their cells, so they are at least as far as this cell's border

```
def structures w =
  let above = w.y - structure_top
  in if above > 0.3 then above else
  let far = cell_border w + margin
  in if !present (cell w) then far else
  ...
  in min (sd_structure kind q) far
```

Every value returned is a **lower bound** on the true distance, so stepping by it can never pass through a wall.

Two marches and layers

- The scene mixes scales: millimetre fingers next to a 150-unit horizon. One march over everything would take tiny steps everywhere
- So each camera ray is marched **twice**:
  1. the **land** alone (terrain, water, buildings) over the whole ray, with its own tricks
  2. the **local scene** (wizard, staff, rock, glows) only inside a sphere around him, up to where the land was hit
- Every part belongs to a **layer** (a bit in a mask): the same scene functions serve the camera, the shadows (which parts cast) and the debug toggles in the drawer
- **Bounding volumes** everywhere: far from a capsule around the figure, return the capsule's distance and skip all the parts inside

Futhark: a functional language for GPUs

- Pure, statically typed, array-oriented (like a small ML or Haskell)
- Programs are written with `map`, `reduce`, `scan`, `filter`, `tabulate`; the compiler turns them into GPU kernels
- Records and tuples compile to registers: a `vec3` is just three floats
- A frame is one expression:

```
tabulate (w * h) (\i ->
  let (x, y) = (i % w, i / w)
  in pack (pixel prm x y))
```

- Backends: C, multicore, CUDA, OpenCL, and (on a development branch) **WebGPU**, generating WGSL shaders plus a WebAssembly host
- The same source also compiles to a **CPU program**: byte-exact reference renders, used to check that refactors change nothing
- Functions are inlined by default; `#[noinline]` keeps one as a real GPU function. This turned out to matter a lot (later)

From Futhark to the screen, without the CPU

1. Futhark's WebGPU backend writes the frame into a GPU **storage buffer** of packed pixels
2. The harness (TypeScript) gets that buffer's handle out of the WebAssembly module
3. One `copyBufferToTexture` puts it into the canvas

No round trip through the CPU, no extra drawing pass. Futhark creates its own GPU device; the harness captures it by wrapping `requestDevice` during initialisation.

- **Uniforms**: 80 parameters (colours, sizes, glow strengths) from a generated drawer, passed as a Futhark **record**. The drawer, the record type and its defaults are all generated from one JSON schema
- **Compile-time parameters** (step counts) rebuild the shader: the dev server rebuilds on save, content-addressed by the hash of every input, and hot-swaps the build into the open page

Interactive, while staying pure

- All state lives in TypeScript: camera, free flight (velocity, drag, banking), spells in flight, cooldowns
- Each frame it is passed in as plain data: a **drive** record (where he is, how he accelerates, the staff gesture) and a flat array of spells
- The renderer stays a pure function of (time, parameters, state)
- **Inverse kinematics** (two-bone, closed form) places elbows so the hands reach the staff and the pointing target
- **Collisions** need the scene's distance on the CPU side. A second tiny entry point, `probe`, evaluates it at a few points (spheres around the rock, each spell)
- Its 100-odd bytes are copied out and read back **asynchronously**, a frame or two late, so rendering never waits on it
- Spells are glows computed in closed form per pixel (distance from the ray to a segment), so they cost no extra marching

A WebGL fallback from the same program

- WebGL has no compute shaders, and Futhark only generates compute kernels
- But the per-pixel work sits in two GPU functions that take only scalars. So each build gets a **second entry point**: a fragment shader that calls them, fed from a uniform block
- **naga** (the WGSL translator from the Rust wgpu project) turns it into GLSL for WebGL2
- The page picks WebGPU, or falls back if the browser has none, or its adapter fails, without looping or giving up silently
- Shader compiles block the page in WebGL; compiling first in a throwaway **Worker** fills the browser’s shader cache, so the real compile then takes milliseconds

Where time goes

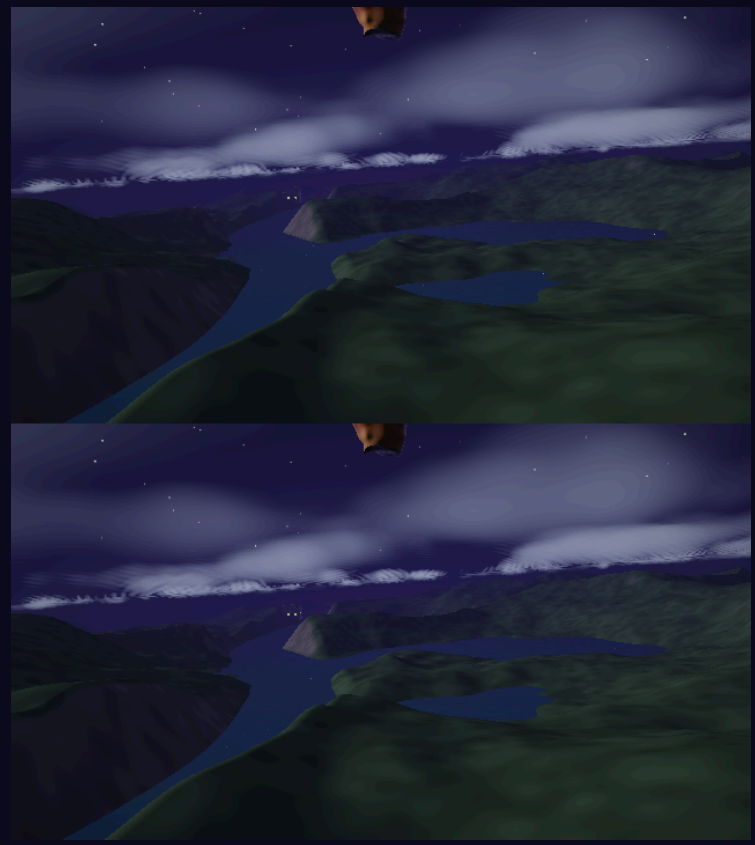
- Measured by **ablation**: build variants with one feature switched off, and time them
- Plus GPU **timestamp queries** around every kernel
- At the start of the last round (22.4 ms a frame):
  - land march and its shading: 11 ms
  - local march (wizard, glows): 5 ms
  - shadows 3, clouds 2, AO 0.5

Biggest wins, roughly in order:

- looser surface tolerance on distant terrain, matched to pixel size
- bounding groups of parts (face, hat, beard)
- over-relaxed sphere tracing (step 1.4× and back off on overshoot)
- not re-evaluating buildings at every terrain step: a bound found earlier stays valid, minus the distance travelled
- shadows from a simplified stand-in figure

Trading pixels for milliseconds

- A pixel at distance t is $\approx 0.0008t$ wide: a surface found within $0.0015t$ is within two pixels' depth, so silhouettes don't move
- Content trade-offs, measured and reversible: one fewer terrain octave, 16 cloud samples, a 150-unit horizon
- Quality presets are separate builds; **Auto** watches the frame rate and swaps builds while the old one keeps drawing



Hit tolerance 0.0004 t (top) vs 0.0015 t (bottom): 1.7 ms faster, visually the same.

Shader size: the hidden cost

- Inlining everything made a 2.4 MB shader; Chrome compiles each **pipeline** (kernel) at load, slowly, blocking
- Naive `#[noinline]` made it **worse**: GPU functions get every record field as a parameter, and the pose has 200
- What worked: **project** each function’s inputs into a small record; fewer kernels holding the shading; asynchronous pipeline creation
- 2.4 → 1.6 MB; time to first frame about halved

```
type sky_in = { horizon, zenith, moon,
                moon_size, ..., secs }

#[noinline]
def sky_colour (sk: sky_in) (rd: vec3) = ...

def sky pose rd = sky_colour (sky_in_of pose)
rd
```

The wrapper keeps every call site unchanged; only the GPU function’s signature shrinks.

Anti-aliasing as a second pass



- **FXAA**: look at each pixel’s neighbours; where contrast suggests an edge, blend across it. Cheap (0.4 ms), slightly soft
- **Edge supersampling**: find pixels whose hit distance jumps or whose contrast is high, compact them into a list (`filter`), and trace 4 more rays only for those. Sharp, 11 ms

Benchmarking honestly on a laptop

- An **A/A test** (two identical builds) shows the noise: frame time varies by 0.25 ms between rounds
- The laptop runs 1 ms faster cold, drifts for a minute, then holds at its power limit. Fans at 100% changed nothing: power-bound, not heat-bound
- So: warm up first, then interleave builds **A B B A** so drift hits each equally, and report a mean difference with a 95% confidence interval

Difference to detect	Rounds needed
0.5 ms	3
0.25 ms	9
0.1 ms	55

95% confidence, 80% power, paired rounds of 12 frames at 1080p. Without this, “optimisations” of 0.2 ms are coin flips.

Bugs found along the way

- **Bools passed to kernels were sometimes true when false**: the runtime copied 1 byte into a 4-byte slot of uninitialised memory. Fixed by patching `malloc` to `calloc` in the generated C
- **Function names with a ‘z’** broke GPU calls: the name escaping differed between definition and call site (fixed upstream)
- `x ** 2` **with negative x** is NaN in WGSL: one shockwave ring turned the screen white
- **Empty arrays** bound as too-small buffers; **Asyncify’s 4 KB stack** overflowed on a mid-frame readback
- **Chrome compiling pipelines on its GPU process’s main thread**, freezing other canvases for seconds

Most were found by rendering on the CPU and the GPU and comparing, or by bisecting with build variants.

Takeaways

- **Distance functions** make geometry, shadows, AO and collisions all fall out of one function
- **Correct bounds** are the core of ray-marching performance: every shortcut must return a value that is never larger than the true distance
- A **pure renderer** with state passed in as data is easy to test: byte-exact CPU references catch regressions
- On the GPU, **code size** and **kernel count** matter as much as arithmetic
- **Measure** before believing: ablations, timestamps, A/A tests, confidence intervals

Source: github.com/badly-drawn-wizards/wizard-vibes · Live: badly-drawn-wizards.github.io/wizard-vibes